

CHAPTER – 4

FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES ANALYSIS

4.1 Introduction

The **ShopSphere Online E-Commerce Database Management System** is designed to manage the information related to customers, sellers, products, categories, orders, payments, deliveries, inventory, and customer feedback. Functional dependencies are used to identify how one attribute determines other attributes within an entity. Proper identification of functional dependencies helps in database normalization, reduces redundant data, and maintains data consistency. A well-designed database minimizes duplicate data by storing each piece of information only once. It also prevents insertion, update, and deletion anomalies, thereby improving the reliability and efficiency of the database system. The following sections explain the functional dependencies, redundancy analysis, and data anomalies of each entity in the ShopSphere database.

1. CUSTOMER

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address, Registration_Date

Redundancy Analysis

The Customer table stores each customer's information only once using a unique **Customer_ID**. Customer details are not repeated in other tables. Other entities, such as the Order table, use **Customer_ID** as a foreign key to establish relationships. Hence, duplicate customer records are avoided.

Data Anomalies

- **Insertion Anomaly:**
A new customer can be registered without placing any order. Therefore, no insertion anomaly exists.
- **Update Anomaly:**
If a customer changes the phone number or address, it is updated only in the Customer table. No duplicate updates are required.
- **Delete Anomaly:**
Deleting an order does not remove customer information because customer records are stored independently.

2. SELLER

Functional Dependency

Seller_ID → **Seller_Name, Email, Contact_Number, Business_Type**

Redundancy Analysis

Seller information is stored only once in the Seller table. Products are linked using **Seller_ID**, which avoids repeating seller information for every product.

Data Anomalies

- **Insertion Anomaly:**
A seller can register without adding any products.
- **Update Anomaly:**
Seller details are updated only in one place.
- **Delete Anomaly:**
Deleting a product does not remove seller information.

3. PRODUCT

Functional Dependency

Product_ID → **Product_Name, Price, Brand, Stock_Quantity, Product_Description**

Redundancy Analysis

Product details are maintained only in the Product table. Other entities refer to products using **Product_ID**, reducing duplicate product information.

Data Anomalies

- **Insertion Anomaly:**
New products can be added before any customer purchases them.
- **Update Anomaly:**
Changes to the product price or description are updated only once.
- **Delete Anomaly:**
Deleting an order does not delete product details.

4. CATEGORY

Functional Dependency

Category_ID → **Category_Name, Description**

Redundancy Analysis

Each category is stored only once. Products reference the category using **Category_ID**, preventing repeated category names.

Data Anomalies

- **Insertion Anomaly:**
A category can be created even if no products belong to it.
- **Update Anomaly:**
Updating a category name is performed only once.
- **Delete Anomaly:**
Deleting a product does not remove the category.

5. ORDER

Functional Dependency

Order_ID → Customer_ID, Order_Date, Order_Status, Total_Amount

Redundancy Analysis

The Order table stores only order-related information. Customer details are not duplicated because they are referenced through **Customer_ID**.

Data Anomalies

- **Insertion Anomaly:**
Orders can be created after selecting an existing customer.
- **Update Anomaly:**
Changing the order status affects only the Order table.
- **Delete Anomaly:**
Deleting an order does not remove customer or product information.

6. PAYMENT

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount

Redundancy Analysis

Payment information is stored separately from orders, avoiding duplicate payment details.

Data Anomalies

- **Insertion Anomaly:**
Payment details can be entered after the order is created.
- **Update Anomaly:**
Updating payment status requires changes only in the Payment table.

- **Delete Anomaly:**
Deleting payment information does not delete the order.

7. DELIVERY

Functional Dependency

Delivery_ID → Order_ID, Delivery_Status, Expected_Date, Delivery_Address

Redundancy Analysis

Delivery information is maintained in a separate table, preventing repeated delivery details for each order.

Data Anomalies

- **Insertion Anomaly:**
Delivery information can be added after confirming the order.
- **Update Anomaly:**
Delivery status can be updated independently.
- **Delete Anomaly:**
Deleting delivery details does not affect order information.

8. INVENTORY

Functional Dependency

Inventory_ID → Product_ID, Available_Quantity, Reorder_Level, Last_Updated

Redundancy Analysis

Inventory details are stored separately from product details. The Product table refers to inventory using Product_ID, which avoids duplicate stock records.

Data Anomalies

- **Insertion Anomaly:**
Inventory records can be created when a new product is added.
- **Update Anomaly:**
Changes in stock quantity are updated only in the Inventory table.
- **Delete Anomaly:**
Deleting inventory information does not remove product details.

9. FEEDBACK

Functional Dependency

Feedback_ID → Customer_ID, Product_ID, Rating, Comments, Feedback_Date

Redundancy Analysis

Customer feedback is stored only once for each feedback record. Product and customer details are linked using their respective IDs instead of repeating complete information.

Data Anomalies

- **Insertion Anomaly:**
Feedback can be added after a customer purchases a product.
- **Update Anomaly:**
Editing a feedback comment or rating affects only the Feedback table.
- **Delete Anomaly:**
Deleting feedback does not remove customer or product information.

CONCLUSION

The **ShopSphere Online E-Commerce Database Management System** is designed using appropriate functional dependencies and normalized database tables. Each entity contains only its relevant information, while relationships are maintained using primary and foreign keys. This design minimizes data redundancy and prevents insertion, update, and deletion anomalies. As a result, the database maintains data integrity, consistency, and efficient performance, making it suitable for managing an online e-commerce platform.